

BlockQuiz
Web-Based Block Programming Platform with Auto-Grading for
students

Fabio Plunser



B.Sc. Thesis

Supervisor: Ass. Prof. Dr. Michael Vierhauser
5th May 2026

Abstract

This thesis presents the design and implementation of BlockQuiz, a web-based learning platform for block-based programming exercises aimed at children aged 8-12. The work is motivated by the increasing importance of computational thinking in school curricula and, in particular, by the Austrian subject "Digitale Grundbildung", which creates demand for classroom-ready tools that offer guided learning tasks, immediate feedback, and a deployment model suitable for schools.

Existing systems such as Scratch, Code.org, Microsoft MakeCode, and Blockly Games demonstrate the educational value of block-based programming, but they do not combine all of the requirements that are central to this project. Some of them focus primarily on open-ended creativity, others on hardware programming or cloud-hosted delivery, and only a subset support tightly constrained exercises that can be assessed automatically. BlockQuiz addresses this gap through a teacher-oriented platform for authoring and delivering short programming tasks with clear learning goals, progressive hints, and deterministic auto-grading.

The implementation is based on SvelteKit and Google Blockly and includes a client-side sandbox for running generated code, a grading framework for input/output, turtle, and robot exercises, multilingual exercise content, and a privacy-first deployment model that supports self-hosting in school networks. Particular attention is paid to the authoring workflow for non-programming teachers, the safety limitations of browser-based execution, and the extensibility of the exercise framework.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Ass. Prof. Dr. Michael Vierhauser, for his invaluable guidance, constructive feedback, and continuous support throughout this project. His expertise in software quality and educational technology significantly shaped the direction of this work.

I am also grateful to the Quality Engineering research group at the University of Innsbruck for providing an inspiring academic environment and the resources necessary to complete this thesis.

Special thanks go to everyone who discussed the prototype and provided informal feedback during development, helping to improve the platform's design and functionality.

Finally, I would like to thank my family and friends for their encouragement and patience during the development of this project.

Contents

Abstract	i
Acknowledgments	iii
Contents	v
List of Figures	vii
List of Tables	ix
Declaration	xi
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Goals and Contributions	1
1.3 Target Audience	2
1.4 Scope and Limitations	2
1.5 Document Structure	2
2 Background and Related Work	3
2.1 Block-Based Programming Environments	3
2.1.1 Scratch	3
2.1.2 Microsoft MakeCode	3
2.1.3 Google Blockly	3
2.1.4 Code.org	4
2.1.5 Open Roberta	4
2.1.6 LEGO Mindstorms	4
2.1.7 Comparison Summary	4
2.2 Automated Assessment in Programming Education	5
2.2.1 Test-Based Grading	5
2.2.2 State-Based Grading	5
2.2.3 Benefits of Immediate Feedback	5
2.3 Learning Theory Foundations	6
2.3.1 Constructionism	6
2.3.2 Cognitive Load Theory	6
2.3.3 Age-Appropriate Design	6
2.3.4 Privacy and Data Protection	7
2.3.5 Technical foundations	7

3	Requirements Analysis	9
3.1	Overview	9
3.2	Stakeholders	9
3.3	Functional Requirements	9
3.4	Non Functional Requirements	11
3.4.1	Representative Use Cases	11
4	System Design	13
4.1	Design decisions	13
4.1.1	Block Editor: Google Blockly	13
4.1.2	Execution mode: Client-Side sandbox	13
4.1.3	Database: SQLite with Drizzle ORM	14
4.2	Architectural Overview	14
4.2.1	Data Model	15
4.2.2	Security Considerations	15
5	Implementation	17
5.1	Implementation Scope	17
5.2	Project Structure and Technology Stack	17
5.2.1	Domain Model and Persistence	17
5.2.2	Blockly Integration	18
5.2.3	Execution Pipeline and Trust Boundaries	20
5.2.4	Grading Pipeline	21
5.2.5	Learner Player and Attempt Capture	21
6	Evaluation	23
7	Conclusion	25
	Bibliography	27

List of Figures

4.1	System Architecture Overview	14
-----	--	----

List of Tables

2.1	Comparison of Block-Based Programming Platforms	5
4.1	Block Editor Selection	13
4.2	Code Execution Options	14
4.3	Database Selection	14
5.1	Implemented exercise families in the prototype	18
5.2	Main categories of persisted attempt data	19

Declaration

By my own signature I declare that I produced this work as the sole author, working independently, and that I did not use any sources and aids other than those referenced in the text. All passages borrowed from external sources, verbatim or by content, are explicitly identified as such.

Signed: Date:

1 Introduction

Computational thinking has emerged as a fundamental skill for the 21st century, increasingly recognized as essential for all learners regardless of their future career paths (24; 7). In Austria, this recognition materialized with the introduction of “Digitale Grundbildung” (Digital Basic Education) as a mandatory subject for secondary education in 2022/23, reflecting a broader European trend toward integrating computational concepts into primary and secondary curricula. As a result, schools need programming tools that fit actual classroom conditions: they must be understandable for beginners, manageable for teachers with limited programming experience, and suitable for an institutional context in which privacy, deployment simplicity, and curricular alignment all matter.

1.1 Motivation and Problem Statement

Many established block-based environments already support introductory programming quite well, yet they do not fully address this combination of requirements. Scratch encourages creativity and experimentation, but its open-ended project structure makes it difficult to use for short, deterministically assessable classroom tasks. Microsoft MakeCode offers a highly polished environment and a valuable bridge from blocks to text, but much of its design is centered on hardware integration and project-based exploration. Code.org provides structured learning sequences and strong scaffolding, yet its deployment model and platform boundaries are less aligned with a privacy-first, self-hosted use case in European school settings. Teachers therefore face a practical gap between highly creative block environments on the one hand and the need for tightly guided, easy-to-manage learning tasks on the other.

This thesis addresses that gap through the design and implementation of BlockQuiz, a web-based platform for short block-programming exercises with automatic feedback. The central idea is to combine the accessibility of block-based programming with a constrained exercise model that reduces cognitive load, supports clear learning objectives, and enables deterministic grading. At the same time, the platform is designed for teachers who may not have a strong programming background and for schools that prefer local or self-hosted deployment over dependence on external services.

1.2 Goals and Contributions

The goal of the project is not to replace general-purpose creative environments, but to provide a complementary platform for guided practice. BlockQuiz focuses on short exercises in which the available blocks, the expected interaction, and the grading criteria are defined in advance. This structure allows the platform to deliver immediate and meaningful feedback while also giving teachers a manageable authoring workflow. The thesis therefore contributes a constrained exercise model for beginner-oriented programming tasks, a browser-based sandboxing and grad-

ing architecture, a teacher-oriented content management approach for exercise authoring, and a privacy-conscious deployment model that supports classroom use without relying on third-party cloud services.

1.3 Target Audience

The primary audience of the platform consists of children aged 8–12, a group for whom block-based interaction remains especially appropriate because it removes syntax barriers and keeps attention on logic, sequencing, repetition, and conditional behavior. A second important audience consists of teachers who need to create, organize, and evaluate exercises without having to program the platform itself. Finally, the system also addresses administrators or school IT staff who need a solution that can be deployed and maintained with limited operational complexity.

1.4 Scope and Limitations

The implemented minimum viable product focuses on three exercise types: input/output tasks, turtle-graphics tasks, and grid-based robot tasks, together with a basic course model, multilingual content, and a teacher-facing authoring workflow. The platform emphasizes constrained tasks, client-side sandboxed execution, server-side authoritative grading for authenticated submissions, and Docker-based deployment. It does not attempt to provide a complete learning management system, full classroom analytics, advanced institutional identity management, or a broad range of domain-specific simulators. These omissions are intentional, as the thesis concentrates on the core software of a focused, privacy-conscious block-programming platform.

1.5 Document Structure

The remainder of this thesis follows the development of the platform. Chapter 2 reviews the educational and technical background, discusses related platforms, and situates the work in existing literature on block-based programming and automatic feedback. Chapter 3 derives the requirements from the target group, the classroom context, and the intended deployment model. Chapter 4 then explains the resulting system design and the main architectural decisions. Chapter 5 describes the implementation of the platform in more detail, including the integration of Blockly, the execution sandbox, and the grading framework. Chapter 6 presents a technical and heuristic evaluation of the prototype, including limitations and future validation needs. Finally, Chapter 7 summarizes the thesis, reflects on limitations, and identifies directions for future work.

2 Background and Related Work

2.1 Block-Based Programming Environments

Block-based programming environments have become a dominant way of introducing novice learners to programming because they remove syntax errors from the early learning process and make program structure visible through shapes, categories and direct manipulation (23). Rather than asking beginners to memorize keywords and punctuation, they make sequencing, repetition and conditional logic tangible. This is especially relevant for younger learners, for whom the transition from an idea to a formal textual program can otherwise be discouraging.

Furthermore studies of learning via block-based programming suggest that block-based programming can support the development of programming concepts, problem solving, and curricular integration when tasks are appropriately scaffolded (13; 11; 19).

2.1.1 Scratch

Scratch, developed at the MIT Media Lab, is one of the most influential systems in this area (18). Its educational strength lies in the way it connects programming to storytelling, games, animation and personally meaningful projects. For many learners, this creative openness is precisely what makes programming engaging. At the same time, the very openness that makes Scratch pedagogically powerful can make it difficult to use for tightly defined classroom exercises. Teachers who want to assign short tasks with clear expected outcomes often need more structure than Scratch is designed to provide and the platform does not primarily revolve around deterministic auto-grading.

2.1.2 Microsoft MakeCode

Microsoft MakeCode extends the idea of block-based learning toward hardware and project based exploration. It offers polished editors, simulators and a transition path from blocks to JavaScript or Python. This makes it particularly useful in contexts where physical computing or the blocks-to-text progression is part of the learning goal. However, its overall focus differs from a platform centered on short, tightly constrained exercises. In comparison to the present work, MakeCode is less concerned with teacher-authored assessment tasks and more concerned with broad exploratory creation in hardware-oriented settings.

2.1.3 Google Blockly

Google Blockly is best understood not as a complete learning platform but as a library for building visual programming interfaces (6). This makes it particularly relevant for custom educational systems. Blockly provides a mature rendering and connection model for blocks, configurable toolboxes, internationalization support, and code generators for multiple target

languages. Because it is a library rather than a fixed educational product, it enables the construction of environments in which the available blocks, the surrounding user interface and the grading logic can be aligned with specific pedagogical goals. For BlockQuiz, Blockly therefore serves as a technical foundation rather than as an end-user product.

2.1.4 Code.org

Code.org demonstrates how strongly structured learning paths can support school teaching. Its courses combine guided exercises, immediate feedback, and clear curricular progression, which makes it an important reference point for guided beginner instruction. In contrast to more open creative environments, Code.org is designed around sequenced lessons and classroom management. This makes it particularly relevant as a comparison for BlockQuiz, since both systems emphasize guided tasks over unrestricted project work. However, Code.org is primarily delivered as a centralized online platform and is less focused on self-hosted, privacy-conscious deployment in local school environments.

2.1.5 Open Roberta

Open Roberta is especially relevant from a European and school-oriented perspective. Developed as a browser-based platform for educational robotics, it was designed to reduce technical barriers for teachers and learners by avoiding complex local installation and configuration steps. Its visual programming model, accessibility for beginners, and compatibility with classroom contexts make it a valuable comparison point for BlockQuiz. At the same time, Open Roberta is centered more strongly on programming robots and boards, whereas BlockQuiz focuses on constrained programming exercises with integrated auto-grading and teacher-authored task sequences.

2.1.6 LEGO Mindstorms

LEGO Mindstorms is one of the best-known robotics platforms in introductory programming education and has played an important role in bringing programming into schools through hands-on, tangible activities. Its strength lies in the connection between software and physical devices, which can be highly motivating for learners. In educational terms, it shares with Open Roberta a strong emphasis on robotics, experimentation, and visible program behavior. However, this hardware-centered approach also distinguishes it from BlockQuiz, whose main purpose is not robot control but the delivery of guided, browser-based programming exercises that can be deployed and assessed without requiring dedicated physical equipment.

2.1.7 Comparison Summary

Taken together, these platforms show that there is no single dominant design for block-based programming education. Scratch foregrounds creativity, while MakeCode, Roberta and LEGO Mindstorms bridge visual and textual programming in hardware-oriented contexts. Code.org offers strongly scaffolded instruction and Blockly acts as an enabling framework. BlockQuiz positions itself with a different focus where constrained exercises, teacher authoring, automatic assessment and privacy-conscious deployment need to coexist.

Table 2.1: Comparison of Block-Based Programming Platforms

Platform	Auto-grading	Teacher CMS	Self-hostable	Open Source
Scratch	No	No	No	Yes
MakeCode	Partial	No	No	Yes
Code.org	Yes	Limited	No	Partial
Blockly	N/A (library)	N/A	Yes	Yes
BlockQuiz	Yes	Yes	Yes	Yes

2.2 Automated Assessment in Programming Education

Automated assessment of programming exercises has been an active research area since the 1960s, with early classroom systems already described by Hollingsworth, Naur, and Forsythe and Wirth (9; 15; 5); later surveys document the continued development of this research area (1; 3; 10; 14). Modern auto-graders employ various strategies to evaluate student code, each with distinct advantages and limitations.

2.2.1 Test-Based Grading

The most common approach executes student code against a set of test cases (3; 1), comparing actual output to expected outputs. This is well suited to input/output exercises in which correctness can be defined through a clear mapping from input to output. In practice, such grading must deal with issues such as visible versus hidden tests, normalization of whitespace or upper/lower case and the distinction between purely summative checking and formative feedback. The systematic review by Messer et al. shows that this form of grading remains the dominant approach in automated programming assessment because it is reliable for constrained tasks and scales well to large cohorts (14).

2.2.2 State-Based Grading

Not all beginner tasks produce meaningful textual output. In graphical or simulation-based exercises, correctness is often better captured through the final state of the system or through the sequence of actions that produced the state. Turtle graphics are a good example: a learner may arrive at the same picture through multiple valid block sequences, which means that grading can be based either on the final geometric state or a more prescriptive comparison of executed commands (12). For this reason, BlockQuiz combines simple command-based and state-based grading strategies, depending on the intended learning objective.

2.2.3 Benefits of Immediate Feedback

The pedagogical importance of immediate feedback is well established in the literature on formative assessment (2; 8; 20). Fast feedback allows learners to revise their understanding while the problem is still active in working memory, and it reduces the delay between action and consequence that often weakens learning in traditional classroom assessment. In the context of beginner programming, this is especially important because misconceptions about loops, sequencing or conditions can often be addressed most effectively when the learner can try again immediately.

2.3 Learning Theory Foundations

The design of BlockQuiz is informed by several well-established educational theories. These theories do not determine the implementation directly, but they provide a conceptual framework for understanding why constrained block-based tasks, progressive hints and visual feedback are appropriate design choices.

2.3.1 Constructionism

Seymour Papert’s constructionism argues that learning becomes especially powerful when students actively construct artifacts that are meaningful to them (16). Programming can therefore be understood not merely as a way of issuing instructions to a machine, but as a medium through which learners explore ideas, observe consequences and build mental models. Although BlockQuiz focuses on shorter and more guided tasks than open creative environments, it still adopts the constructionist insight that programming is most engaging when learners can see the effects of their decisions directly. Turtle graphics, in particular, make this visible by connecting code to movement and shape.

2.3.2 Cognitive Load Theory

Cognitive Load Theory distinguishes between the inherent difficulty of a task, extraneous load introduced by presentation or interface design and beneficial load associated with meaningful learning activity (21). One of the main strengths of block-based programming is that it reduces extraneous load by eliminating syntax errors and making valid composition visible. BlockQuiz extends this idea by constraining the toolbox on a per-exercise basis. Instead of exposing a large library of irrelevant blocks, the system presents only the constructs that are needed for a given task, which supports focused learning and reduces distraction.

Zone of Proximal Development

Vygotsky’s Zone of Proximal Development describes the range in which learners can complete a task with guidance but not yet entirely on their own (22). This idea is relevant for platforms that provide progressive support rather than all-or-nothing assessment. In BlockQuiz, the use of staged hints, starter blocks and ordered task progression is intended to keep exercises within that zone. The platform does not merely judge whether an answer is correct, it is also designed to support learners while they move toward a correct solution.

2.3.3 Age-Appropriate Design

Design for children aged 8-12 requires more than simply scaling down an adult programming environment. At this age, many learners are still more comfortable with concrete and visual representation than with abstract symbolic formalisms. Tasks therefore need to connect closely to visible outcomes, especially for younger learners, while still leaving room for gradually more abstract reasoning among older children in the age group. This makes a mixture of turtle-graphics tasks and simple input/output problems an appropriate scope for the platform.

Research in child-computer interaction further emphasizes the importance of large interaction targets, concise text, clear feedback and forgiving interfaces (17). These principles translate directly into the design of a block-based learning platform. A child should be able to understand what an exercise asks for without encountering unnecessary textual complexity, should be able to manipulate the blocks without fine motor frustration, and should receive clear feedback after

each attempt. Readability and immediate visual response are therefore not cosmetic concerns but core usability requirements.

2.3.4 Privacy and Data Protection

Privacy is a particularly important design consideration when educational software is used with children. Within the European Union, the GDPR places strict limits on the processing of personal data and emphasizes data minimization, purpose limitation and privacy by design. For school software, this means that architectural choices such as account models, storage location and reliance on external services are not only technical decisions but also institutional and legal ones (4).

BlockQuiz adopts a privacy-by-design stance by favoring guest operation, local-progress storage and self-hosted deployment. This does not remove every compliance consideration, but it reduces the amount of personal data that must be collected and transferred. In comparison to cloud-centered educational platforms, this approach makes it easier to imagine deployment in school networks where data sovereignty and operational control are important. Currently, user accounts store a name, email address, role and active status on the local server. Attempts store exercise progress and grading data; this data is not shared with third-party services by the application.

2.3.5 Technical foundations

SvelteKit and Svelte 5

The technical implementation builds on SvelteKit and Svelte 5, whose component model and reactive primitives are well suited to an application that combines a rich frontend, custom editing interfaces and reusable logic for exercise engines. In the context of this project, the relevant advantage is that reactive state can be kept close to the domain logic while still integrating cleanly with user interface components.

For persistence, the project uses SQLite together with Drizzle ORM. This combination is appropriate for the intended deployment scope because it avoids the operational overhead of a separate database server while still allowing a typed and structured data model. In a school environment or single-site deployment, the simplicity of SQLite is often a practical advantage rather than a limitation.

Finally, containerized deployment with Docker provides a deployment model that is consistent with the overall goals of the system. The platform can be packaged as a self-contained service with minimal installation complexity, which is important in settings where dedicated DevOps expertise cannot be assumed.

3 Requirements Analysis

3.1 Overview

This chapter derives the requirements for BlockQuiz from the intended learner group, the realities of classroom teaching, the desired self-hosted deployment model and the technical consequences of executing learner programs in the browser. The purpose of the requirements analysis is not to produce an exhaustive catalog of all imaginable platform features. Instead, it identifies the set of capabilities that a realistic prototype must provide in order to support guided block-based programming exercises in a school context.

3.2 Stakeholders

The most important stakeholder group consists of learners between the ages of 8 and 12. For them, the platform must present programming tasks in a form that is visually clear, forgiving of mistakes and capable of returning immediate, understandable feedback. Teachers form the second central stakeholder group. Their interest lies less in the technical implementation than in whether they can create, adapt and organize exercises without touching source code. School IT administrators represent a third perspective, since they need a deployment model that is easy to operate, simple to back up and compatible with school privacy expectations. Finally, the thesis itself also imposes a developer-facing perspective: the architecture should remain maintainable, extensible and honest about its security assumptions. The platform serves multiple stakeholder groups with distinct needs:

3.3 Functional Requirements

Exercise Definition (FR-1)

At the center of the platform is the exercise model. Each exercise must have a stable identity together with learner-facing metadata such as title, description, exercise type, topic and difficulty. Because the platform focuses on guided tasks rather than unrestricted creation, an exercise must also define the subset of blocks that is available to the learner. When appropriate, it may include a starter workspace and one or more hints that support gradual progression through the task. Since the platform is intended for bilingual use, exercise content must support both German and English.

Supported Exercise Types (FR-2)

The platform focuses on exercise families that cover complementary learning situations. Input/output tasks provide a way to model simple algorithmic problems with deterministic textual results, while turtle-graphics and robot-grid exercises make program behavior visible through

movement and spatial state. At the same time, the exercise architecture should not be so specialized that the addition of future exercise types would require a complete redesign.

Authoring and Course Organization (FR-3)

Authorized users must be able to create and edit exercises through a teacher-facing interface. They also need a way to combine exercises into ordered courses so that tasks can be used as part of a broader learning path instead of only as isolated examples. Reuse is an important requirement here: the same exercise should be usable in more than one course. Since authoring mistakes can easily lead to confusing learner experiences, the system should also support previewing an exercise before it is published

Execution and Feedback (FR-4)

Learner block programs must be translated into executable code and run inside a restricted environment. The execution model therefore needs a controlled API surface through which generated programs can interact with the exercise, while preventing direct access to browser capabilities that are outside the task domain. Because beginner code often contains accidental infinite loops or other non-terminating behavior, the runtime must also enforce time or step limits. After execution, the learner should receive feedback immediately so that the platform supports an iterative learning process.

Auto-Grading (FR-5)

Automatic grading is one of the defining features of BlockQuiz. Input/output exercises therefore require a grading mechanism based on specified test cases, whereas turtle and robot exercises require comparison of command traces, final state, or both. The grading result must at least distinguish between successful and unsuccessful attempts, but a binary pass/fail signal alone is not sufficient for the educational purpose of the platform. The system should therefore present feedback that is more informative and helps learners understand why a solution did or did not satisfy the task.

User Modes and Progress (FR-6)

The platform must support authenticated teacher access for authoring and management tasks, yet the learner side should remain flexible enough to support guest use where possible. This is particularly relevant for privacy-conscious deployment scenarios or informal try-out settings. Exercise progress must be persisted in a way that matches the chosen user mode, whether this means local storage for guest use or server-side persistence for authenticated users.

Internationalization (FR-7)

Internationalization applies both to the general interface and to exercise content. The user interface should support German and English and exercises should be available bilingually with a defined fallback behavior where a translation is incomplete. Since learners interact with programming blocks as part of the same environment, Blockly labels should be localized consistently as well.

3.4 Non Functional Requirements

Security (NFR-1)

Student-generated code must not receive unrestricted access to the document object model, browser storage, or general network functionality. Data crossing trust boundaries, especially authoring input and remote API requests, must be validated carefully. In addition, authoring and administrative features require role-based access control so that privileged functions remain separate from the learner experience.

Privacy (NFR-2)

Privacy requirements follow directly from the intended school context. The system should minimize personal-data collection, support self-hosting inside school infrastructure and remain usable without mandatory reliance on third-party services wherever feasible. These requirements are not peripheral operational details; they shape the architecture and deployment strategy from the start.

Usability (NFR-3)

Since the main learner group has limited prior programming experience, the user interface must remain clear and age-appropriate. This includes constrained toolboxes, concise instructions and feedback that avoids unnecessary technical jargon. Usability is particularly important because the educational value of the platform depends on whether children can stay focused on the programming problem rather than becoming stuck in the interface.

Performance (NFR-4)

Interactive classroom use requires that exercises load quickly and that feedback appears without disruptive delays. The execution sandbox should therefore fail fast when programs exceed defined limits and the application as a whole should remain responsive on typical school hardware.

Maintainability and Extensibility (NFR-5)

The prototype should provide a technical foundation that can be extended after the thesis. This requires a separation between user interface, execution logic, grading and persistence, as well as exercise abstractions that can accommodate future task types. The system should also be reproducible across environments with minimal configuration effort so that deployment and further development do not depend on undocumented local setup steps.

3.4.1 Representative Use Cases

The core value proposition of the platform can be summarized through a small set of representative use cases. The first is the learner workflow: a child opens an exercise, assembles a block program, runs it and receives immediate feedback. The second is the teacher workflow: a teacher defines task text, available blocks, hints and grading configuration for a new exercise. The third is course composition, in which a teacher selects existing exercises and arranges them into an ordered sequence. The fourth is local deployment, where an administrator sets up the application without depending on external cloud infrastructure. Together, these use cases capture the intended prototype scope. More advanced scenarios, such as deep analytics or integration with institutional learning-management systems, remain outside the current thesis.

4 System Design

4.1 Design decisions

This chapter explains the principal architectural choices behind BlockQuiz. The focus is on decisions that strongly influence the prototype: the visual editor technology, the execution model, the persistence layer and the separation of concerns between exercise definition, execution, grading and presentation.

4.1.1 Block Editor: Google Blockly

Table 4.1: Block Editor Selection

Option	Pros	Cons
Google Blockly	Mature, documented, accessible, customizable toolbox	Larger bundle size
Custom implementation	Full control, smaller size	Development time, maintenance burden
Scratch Blocks	Familiar Scratch aesthetics	Less flexible, tied to Scratch ecosystem

Blockly was selected because it offers the best balance between maturity and adaptability. Its toolbox model directly supports the pedagogical goal of restricting the available blocks on a per-exercise basis, which helps reduce unnecessary complexity for novice learners. The same integration can also be reused in both the learner-facing player and the teacher-facing CMS. A custom implementation would have provided greater control, but the additional development effort would have been difficult to justify within the scope of the thesis. Scratch Blocks, while visually familiar, would have constrained the platform more strongly to an ecosystem that is not primarily designed for this kind of custom guided-learning workflow.

4.1.2 Execution mode: Client-Side sandbox

The execution model is intentionally optimized for classroom responsiveness and low deployment complexity. A client-side iframe sandbox therefore represents a pragmatic compromise. It allows learners to receive immediate feedback without requiring the server to execute every submission, which keeps the overall system lighter and easier to deploy. At the same time, this decision does not remove all security concerns. The thesis therefore treats client-side execution as a risk-managed prototype choice rather than as a perfect isolation boundary. Since authenticated submissions can additionally be re-executed on the server before they are stored, the design combines responsive local interaction with a more trustworthy persistence path.

Table 4.2: Code Execution Options

Option	Pros	Cons
Client-side (iframe sandbox)	No server load, immediate response	Limited isolation compared to server-side, potential security risks
Server-side (VM/Container)	Better isolation	Added complexity, latency
WebAssembly sandbox	Best isolation	Significant development effort

4.1.3 Database: SQLite with Drizzle ORM

Table 4.3: Database Selection

Option	Pros	Cons
SQLite	No setup, embedded, portable	Single-writer limitation
PostgreSQL	Scalable, concurrent writes	Requires server, more complex

SQLite is well suited to a prototype and to modest self-hosted school deployments because it keeps operational overhead low. When combined with Drizzle ORM, it also provides a typesafe development workflow and explicit schema management. A more scalable database system would only become necessary if the platform were extended beyond the intended deployment model.

4.2 Architectural Overview

The system follows a layered architecture in which the user interface, exercise logic, grading and persistence remain conceptually separate. This separation is visible in the code base: Svelte components are primarily responsible for presentation, whereas execution, grading, typing and persistence are handled in dedicated modules. The result is not only easier to reason about technically, but also easier to explain as part of the thesis.

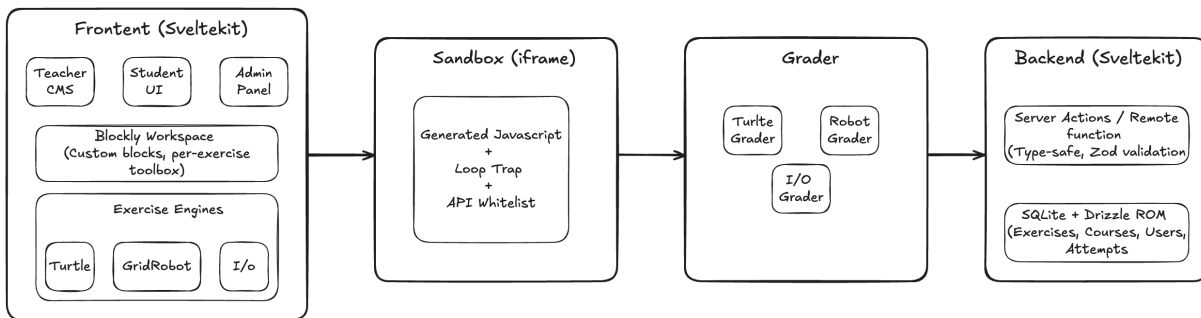


Figure 4.1: System Architecture Overview

Frontend Layer

The frontend is implemented with SvelteKit and primarily supports two interaction contexts. The learner interface presents an exercise together with Blockly workspace, a visual or textual output

area and the resulting feedback. The teacher interface supports exercise authoring, review and course composition. The current implementation also contains additional management views, such as log inspection and user administration, but these play a secondary role in the argument of the thesis. The central design focus remainse the learner workflow and the teacher-facing authoring process.

Exercise Engines

The exercise layer models the behavior of each supported task type, The Input/Output engine provides controlled input consumption and output collection for text-oriented tasks, while the turtle engine manages position, orientation, pen state and drawing history for visual exercises. Each exercise type is associated with an exercise definition, a Blockly toolbox and generator mapping, a restricted execution API and a grading strategy that matches the exercise semantics.

Sandbox

The sandbox executes generated code in a restricted context. in practical terms, execution takes place in an iframe with restrictive sandbox settings and all communication with the main application occurs to browser capabilities; they can only call the exercise-specific commands exposed by the application. Runtime guards additionally limit execution time and help detect non-terminating behavior. The resulting design does not claim absolute security, but it does establish a clear and trust boundary that is appropriate for the prototype.

Graders

The grading layer evaluates exercise outcome after execution. For Input/Output tasks, this means comparing actual and expected output across defined test cases. For turtle exercises, it means comparing either the executed command trace or the resulting final state. Keeping grading separate from execution and presentation makes the platform easier to extend and allows the grading logic to be discussed independently of the user interface.

4.2.1 Data Model

The data model stays close to the implemented scope of the prototype. Its most important entities are users, exercises, course, relations between courses and exercises, and attempt or progress records. users store account and role information wherever authenticated access is needed. Exercises contain the task definition and grading configuration. Courses organize exercises into ordered learning sequences, and the course-exercises relations capture both ordering and reuse. Attempts or progress records connect learners to exercises and preserve the outcome of their interaction with the platform. The repository also contains supporting entities such as audit logs, version tables and translations. These are part of the broader application context, but within the scope of the thesis they remain secondary to the core educational entities of exercises, courses, users and attempts.

4.2.2 Security Considerations

The main threats considered in the design are accidental or malicious infinite loops, unintended access to browser capabilities from learner code, and unauthorized access to protected authoring features. The response to these threats is deliberately layered. Iframe-based sandboxing limits the execution context, command whitelisting constrains the API available to generated programs and timeout or step-based execution limits keep programs from running indefinitely. In parallel,

validation of application input and API payloads protects the wider application, while role-based authorization separates learner, authoring and management privileges. These measures reduce risk for the intended educational prototype but they do not justify absolute security claims.

5 Implementation

5.1 Implementation Scope

The implementation prototype covers the full learner flow, the teacher-facing authoring workflow and the technical infrastructure required for sandboxed execution and automatic grading. The system is broad enough to demonstrate the architecture described in 4 as a coherent whole rather than as a collection of isolated components. In practice, the strongest parts of the code base are the submission pipeline, the grading logic, the typed exercise model and the shared player used by both guest and authenticated learners.

The implementation is organized as a SvelteKit application with TypeScript. The frontend uses Svelte5 components for the learner player, the teacher-facing CMS and the Blockly integration. Persistence is handled through SQLite with Drizzle ORM, while authentication, sessions and password reset are provided through Better Auth. Development and verification are performed through bun-based commands and the same runtime choice is also reflected in deployment through the Bun adapter, the **bun:sqlite** database client and the isolated Bun subprocess that performs server-side code execution.

5.2 Project Structure and Technology Stack

The repository is divided into route components, reusable interface components, typed domain models, grading logic, sandbox infrastructure, remote functions and database access. This separation is visible in the implementation. The learner player and the CMS live in Svelte route components, while exercise normalization, grading, authoritative re-execution and persistence are handled in dedicated library modules.

This split proved useful because the project combines several concerns that would otherwise become tightly coupled: visual programming, learner interaction, runtime control, server-side persistence and authorization. Using SvelteKit for both interface and server integration simplified this boundary, since the same framework can host route components and typed server-side commands. At the same time, the more critical logic such as grading and sandbox coordination is kept outside user-interface files, which improves testability.

The final stack also reflects the deployment goal of a privacy-conscious, self-hostable school system. The application runs without a separate database server, does not depend on third-party APIs for its core functionality and can be shipped as a single containerized web application.

5.2.1 Domain Model and Persistence

One of the most important implementation tasks was turning the originally broad platform idea into a concrete and typed exercise model. In the final prototype, the central runtime model implements three exercise families: **io**, **turtle** and **robot**. Each type has a dedicated config-

Table 5.1: Implemented exercise families in the prototype

Type	Learner interaction	Grading basis	Current maturity
io	Text-oriented tasks with generated JavaScript reading input and producing output	Visible and hidden stdin/stdout tests with configurable normalization	Mature and used as a main exercises family
turtle	Visual movement and drawing on a two-dimensional canvas	Command replay, target checks, final stage and path comparison	Mature and central to the prototype argument

uration and grading strategy, while shared learner-facing properties such as localized content, hints, toolbox restrictions and starter workspaces remain available across all types.

The resulting model is stricter than a generic JSON-based exercise description. For example, io exercises define normalization rules and explicit input and output test cases, where turtle and robo exercises use canvas or grid configuration together with structured state-, command-, or path-based checks. In practice, this made it easier to keep authoring, grading, and persistence consistent, because each exercise type has a clear set of expected fields.

At the same time, the implementation keeps a compatibility-oriented **config** object in addition to the typed branches. This is a pragmatic engineering compromise. It allows generic CMS and player components to access shared properties such as toolbox selection, starter XML, or hints throughout a stable shape, even though the canonical runtime representation is type-specific. The result is slightly less pure than a fully separated model but it reduces UI duplication and kept the authoring surface manageable within thesis scope.

The same emphasis on typed structure appears in persistence. The database schema contains the expected educational entities of users, courses, exercises, course-exercise relations and attempts, but it also stores validation snapshots, exercise version, translations and audit-related information. For the thesis, the most interesting table is the attempt record, because attempts form the common interface between learner interaction, grading, analytics and export.

Instead of storing only a score and a timestamp, the prototype persists the replayable learner artifact: workspace XML, generated code, serialized grading result, locale, hint events, and additional analytics metadata. This makes it possible to reconstruct what a learner submitted and not merely what score they received.

Another important detail is publish validation. The final implementation does not allow exercise publication to remain an informal convention. Instead, exercises are validated against concrete conditions such as bilingual title and description, parseable starter XML, presence of test cases and at least one hidden test. This is not a complete formal verification of exercise quality, but it creates a meaningful boundary between authoring flexibility and learner-facing reliability.

5.2.2 Blockly Integration

Blockly is integrated as the visual programming editor for both learners and teachers. In the learner player, the workspace is configured dynamically from the current exercise definition. The available blocks are not static; instead, the toolbox is assembled at runtime from built-in categories such as logic, loops, math and text together with engine-specific blocks for turtle or

Table 5.2: Main categories of persisted attempt data

Category	Stored values	Purpose
Identity and actor	Attempt id, exercise id, actor type and either authenticated user id or guest client id	
Replayable artifact	Workspace XML and generated JavaScript code	Supports resotation, inspection, export and later re-display of the learner state
Grading result	Serialized result object, score, pass/fail status and timestamp	Preserves the immediate grading outcome without having to recompute it for every view
Context metadata	Locale, hint reveal events, and analytics JSON	Provides a basis for later reporting and for understanding how the learner interacted with the task.

robot behavior. This directly implements the didactic idea of constrained toolboxes discussed earlier in the thesis.

The block integration is not merely visual. Custom block generators emit JavaScript calls such as `api.move(...)`, `api.turn(...)`, or `api.collect(...)`, which become the contract between Blockly and the sandbox runtime. This was an important design choice, because it gives the generated programs a narrow and exercise-specific execution surface instead of unrestricted access to browser objects. The same block definitions can therefore be reused in the learner player, the CMS preview and the grading-related execution paths. The robot exercise type illustrates how this contract can be specialized for a different didactic goal. Where the turtle exercises expose parameterized `move N steps` and `turn N degrees` blocks suitable for free drawing, the robot toolbox uses fixed-action blocks such as *step forward*, *one cell*, *turn left*, and *turn right*. These blocks call the same underlying engine API but remove the need for learners to reason about arbitrary numeric arguments while solving grid puzzles. Together with the optional `collect` block, they make the robot tasks feel like a distinct puzzle activity rather than a numeric variant of the turtle exercises.

The same Blockly component is reused in the CMS editor. Teachers can choose blocks, define starter workspace and preview how an exercise appears from the learner perspective. The implementation stores and restores Blockly workspaces as XML, which allows a starter configuration to be reused consistently between authoring and learner execution. For visual exercise types, Blockly categories combine generic logic and control blocks with engine-specific movement blocks. In practice, the Blockly integration also required several smaller engineering adjustments that are worth documenting. The workspace component adds accessibility attributes after injection so that the generated DOM is exposed as a labelled region, not just as an opaque graphics surface. In addition, a `MutationObserver` is used to keep the flyout scrollbar behavior stable when toolbox visibility changes. These details are easy to omit in a broad design description, but they matter for turning a library integration into a usable web application.

One practical advantage of this design is that the editor remains largely data-driven. Exercise definitions determine both the learner experience and the authoring preview, which reduces duplication between CMS and player code. A typical `io` exercise can expose only arithmetic, text output and control flow blocks, whereas a turtle exercise can offer movement and pen blocks with a smaller set of generic constructs. In both cases, the learner sees only the blocks that are

relevant for the current ask.

5.2.3 Execution Pipeline and Trust Boundaries

The submission pipeline is the strongest technical part of the prototype because it connects editing, execution, grading and persistence through explicit interfaces. From the learner's perspective this appears as a simple *run* or *check* interaction, but the implementation actually performs a sequence of coordinated steps:

1. The Blockly workspace is translated into JavaScript code.
2. The player creates a sandbox request containing the code, exercise type allowed API methods, runtime limits and optional input data.
3. A hidden iframe receives this request through a `MessageChannel` and executes the code in an isolated browser context.
4. The sandbox returns a structured execution trace containing command logs, output streams, timing data and, for visual tasks, the final state.
5. The appropriate grader converts this trace into a uniform grading result.
6. For authenticated submissions, the server validates the shape of the submitted grading object, re-executes the code authoritatively and only then stores the attempt in the database.

This pipeline is intentionally layered. The player is responsible for user interaction, the sandbox is responsible for execution, the graders are responsible for correctness evaluation, and the server-side remote function is responsible for trust-sensitive persistence. This separation makes it easier to reason about failures and avoids hiding core logic in UI components.

The sandbox implementation is intentionally generic across exercise types. The hidden iframe is created with restrictive sandbox settings and communicates with the main application only through typed messages. Each execution request includes the allowed API methods, timeout, maximum command count and maximum loop iterations. Loops in the generated code are instrumented before execution, so the runtime can interrupt programs that would otherwise run indefinitely.

This design provides two practical benefits. First, it keeps the learner interface responsive even when a program fails or loops unexpectedly. Second, it establishes a clearer trust boundary than direct local execution in UI components. The browser-side sandbox is still a pragmatic prototype solution, not a formally strong isolation boundary, but it already separates learner code from the main interface and blocks many browser APIs that are irrelevant or dangerous in this context.

An important extension of this idea is the authoritative execution path on the server side. When an authenticated learner submits an attempt, the server does not simply trust the client-reported score. Instead, it validates the structure of the submitted result and then re-executes the generated code inside an isolated Bun subprocess worker. The worker has no access to the database, the session, or any application module. It receives only the code, the exercise type and the runtime limits over standard input and reports the resulting trace over standard output. Inside the worker, the actual code runs in a `node:vm` context that enforces per-execution time and loop limits and the parent process additionally enforces a hard kill timeout and a maximum outputs size. The grading result that is stored in the database therefore comes from this re-execution rather than from the client-reported value. This is one of the most important implementation choices of the project because it separates immediate learner feedback from trusted persistence at both the API and the operating-system level.

5.2.4 Grading Pipeline

The grading pipeline differs by exercise type, but all graders return the same result structure: pass/fail status, integer percentage score, number of passed test, total number of tests and per-test feedback. This common format was an important implementation goal because it allows the UI, persistence layer and analytics code to remain largely independent of the specific grading strategy.

For `io` exercises, the executor runs the generated program once per test case, injecting the configured standard input and collecting standard output and error output. The grader then normalizes the produced text according to the exercise configuration. The grader then normalizes the produced text according to the exercise configuration. The supported normalization rules include trimming, whitespace collapsing, case handling, line-ending normalization, and decimal separator handling. This is more robust than direct string equality and reduces the number of irrelevant mismatches that beginners would otherwise encounter.

For visual exercises, execution produces a command trace that can be replayed on the corresponding engine. Grading then evaluates the outcome against target, state, command, path, or collect tests. A deliberate implementation decision was to grade the replayed command semantics rather than comparing rendered pixels. This keeps the grader deterministic, easier to test and less dependent on rendering details. In other words, the system asks whether the learner reached the intended logical state, not whether two canvases happen to look identical the pixel level. The current implementation also distinguishes carefully between visible and hidden test. Visible tests can expose expected and actual values to learners, while hidden tests contribute to the score without revealing complete solution information. This is especially important for `io` tasks, where showing all hidden examples would make it easier to overfit a program to the public cases. The grading and submission code therefore treats hidden tests not merely as a UI concern, but as part of the trusted exercise definition.

5.2.5 Learner Player and Attempt Capture

The learner-facing course player presents on exercise at a time, maintains progress within the course and records attempts together with additional metadata. The design intentionally emphasizes focus: the learner sees a task description, as single Blockly workspace, and a results area for execution and grading feedback. Course navigation is still present, but it does not dominate the screen.

On wide screens, these three areas are arranged side by side as a three-column grid that mirrors the conceptual structure of the activity. On smaller screens, the same three areas are exposed as a tab strip with *Task*, *Blocks* and *Run* buttons, while the underlying components remain mounted so that the execution state and grading feedback survive tab changes. This matters in practice because the platform must remain usable on tablets, which are common in primary school classrooms and would otherwise force the learner to scroll between editing and running.

Each submission stores the Blockly workspace XML, the generated code, the grading result, the selected locale and structured metadata such as hint usage and test counts. This is more detailed than a conventional quiz record. The platform does not only preserve whether a learner passed, it preserves what was submitted and under which conditions. This makes later review and export more meaningful and provides a better basis for future teacher-facing analytics. The player also restores stored workspaces state when learners return to an exercise. This is especially relevant for guest mode, where continuity must be preserved without relying on an account. The course-level player therefore does not hard-code a persistence method. Instead, it accepts

6 Evaluation

7 Conclusion

Bibliography

- [1] Kirsti M. Ala-Mutka. A survey of automated assessment approaches for programming assignments. *Computer Science Education*, 15(2):83–102, 2005. doi: 10.1080/08993400500150747.
- [2] Paul Black and Dylan Wiliam. Assessment and classroom learning. *Assessment in Education: Principles, Policy & Practice*, 5(1):7–74, 1998. doi: 10.1080/0969595980050102.
- [3] Christopher Douce, David Livingstone, and James Orwell. Automatic test-based assessment of programming: A review. *Journal on Educational Resources in Computing*, 5(3):4, 2005. doi: 10.1145/1163405.1163409.
- [4] European Union. Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 (General Data Protection Regulation). <https://eur-lex.europa.eu/eli/reg/2016/679/oj>, 2016. OJ L 119, 4.5.2016, pp. 1–88.
- [5] George E. Forsythe and Niklaus Wirth. Automatic grading programs. *Communications of the ACM*, 8(5):275–278, 1965. doi: 10.1145/364914.364937.
- [6] Google. Blockly: A library for building visual programming editors. <https://developers.google.com/blockly>, 2024. Accessed: 2024-12-01.
- [7] Shuchi Grover and Roy Pea. Computational thinking in k–12: A review of the state of the field. *Educational Researcher*, 42(1):38–43, 2013. doi: 10.3102/0013189X12463051.
- [8] John Hattie and Helen Timperley. The power of feedback. *Review of Educational Research*, 77(1):81–112, 2007. doi: 10.3102/003465430298487.
- [9] Jack Hollingsworth. Automatic graders for programming classes. *Communications of the ACM*, 3(10):528–529, 1960. doi: 10.1145/367415.367422.
- [10] Petri Ihantola, Tuukka Ahoniemi, Ville Karavirta, and Otto Seppälä. Review of recent systems for automatic assessment of programming assignments. In *Proceedings of the 10th Koli Calling International Conference on Computing Education Research*, pages 86–93. ACM, 2010. doi: 10.1145/1930464.1930480.
- [11] Filiz Kalelioğlu and Yasemin Gülbahar. The effects of teaching programming via scratch on problem solving skills: A discussion from learners’ perspective. *Informatics in Education*, 13(1):33–50, 2014. doi: 10.15388/infedu.2014.03.
- [12] Lauri Malmi, Ville Karavirta, Ari Korhonen, Jussi Nikander, Otto Seppälä, and Panu Silvasti. Visual algorithm simulation exercise system with automatic assessment: Trakla2. *Informatics in Education*, 3(2):267–288, 2004. doi: 10.15388/infedu.2004.19.

- [13] Orni Meerbaum-Salant, Michal Armoni, and Mordechai Ben-Ari. Learning computer science concepts with scratch. *Computer Science Education*, 23(3):239–264, 2013. doi: 10.1080/08993408.2013.832022.
- [14] Marcus Messer, Neil C. C. Brown, Michael Kölling, and Miaoqing Shi. Automated grading and feedback tools for programming education: A systematic review. *ACM Transactions on Computing Education*, 24(1), February 2024. doi: 10.1145/3636515. URL <https://doi.org/10.1145/3636515>.
- [15] Peter Naur. Automatic grading of students’ algol programming. *BIT*, 4(3):177–188, 1964. doi: 10.1007/BF01956028.
- [16] Seymour Papert. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, New York, 1980.
- [17] Janet C. Read and Mathilde M. Bekker. The nature of child computer interaction. In *Proceedings of the 22nd British HCI Group Annual Conference on People and Computers*, pages 163–170. British Computer Society, 2008.
- [18] Mitchel Resnick, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, and Yasmin Kafai. Scratch: Programming for all. *Communications of the ACM*, 52(11):60–67, 2009. doi: 10.1145/1592761.1592779.
- [19] José-Manuel Sáez-López, Marcos Román-González, and Esteban Vázquez-Cano. Visual programming languages integrated across the curriculum in elementary school: A two year case study using scratch in five schools. *Computers & Education*, 97:129–141, 2016. doi: 10.1016/j.compedu.2016.03.003.
- [20] Valerie J. Shute. Focus on formative feedback. *Review of Educational Research*, 78(1): 153–189, 2008. doi: 10.3102/0034654307313795.
- [21] John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988. doi: 10.1207/s15516709cog1202_4.
- [22] L. S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, Cambridge, MA, 1980.
- [23] David Weintrop and Uri Wilensky. To block or not to block, that is the question: Students’ perceptions of blocks-based programming. In *Proceedings of the 14th International Conference on Interaction Design and Children*, pages 199–208. ACM, 2015. doi: 10.1145/2771839.2771860.
- [24] Jeannette M. Wing. Computational thinking. *Communications of the ACM*, 49(3):33–35, 2006. doi: 10.1145/1118178.1118215.